

Lab3-SIFT 实验报告

1 实验概览

本次实验主要围绕 SIFT 算法的实现，以及与 OpenCV 中的 SIFT 算法进行对比展开。实验核心包含两大模块，具体内容如下：

1.1 SIFT 算法的实现，涵盖四个关键步骤：

- 构建高斯金字塔，并进行多尺度 Harris 角点检测
- 计算 SIFT 描述子
- 特征点匹配
- 找到最佳匹配点对并绘制匹配结果

1.2 与 OpenCV 中的 SIFT 算法进行对比，分析两者在匹配效果与效率上的差异

2 解决思路及核心步骤

2.1 构建高斯金字塔与多尺度 Harris 角点检测

2.1.1 构建高斯金字塔

构建高斯金字塔时，这个函数以输入图像作为金字塔的底层，先将其存入列表。接着通过循环，从第 1 层到指定的金字塔层数，逐层生成上层图像。每次循环时，根据设定的缩放比例（默认 0.5）计算下一层图像的宽和高，再使用 Lanczos4 插值方法对当前层图像进行下采样，得到尺寸缩小的新图像，并存入列表。最终返回的列表包含了从底层到顶层、尺寸依次按比例缩小的所有金字塔图像，为后续多尺度特征检测提供不同分辨率的图像数据。

```
1 def pyramid(input_image, pyramid_levels, scale_ratio=0.5):
2     pyramid_list = [input_image]
3     current_frame = input_image
4     # 遍历生成金字塔各层图像
5     for level_idx in range(1, pyramid_levels):
6         # 计算下采样后图像尺寸
7         new_width_dim = int(current_frame.shape[1] * scale_ratio)
8         new_height_dim = int(current_frame.shape[0] * scale_ratio)
9         # Lanczos4插值方法
10        current_frame = cv2.resize(
11            src=current_frame,
12            dsize=(new_width_dim, new_height_dim),
13            interpolation=cv2.INTER_LANCZOS4
14        )
15        pyramid_list.append(current_frame)
16    return pyramid_list
```

2.1.2 多尺度 Harris 角点检测

该步骤以高斯金字塔为基础，实现了非极大值抑制功能，以特征响应图 ‘response_map’ 和窗口大小 ‘win_size’ 为输入，通过遍历响应图的每个像素，提取以其为中心的局部窗口，仅保留窗口内响应值最大的像素，最终输出筛选后的 ‘suppressed_map’。该操作可过滤局部区域内的次要特征点，减少冗余，是计算机视觉中角点检测、特征提取等任务里筛选显著特征点的关键步骤，能有效提升后续特征匹配、跟踪的精度与效率。

```
1 def suppression(response_map, win_size=3):
2     suppressed_map = np.zeros_like(response_map)
3     # 窗口遍历
4     half_win = win_size // 2
5     for row in range(half_win, response_map.shape[0] - half_win):
6         for col in range(half_win, response_map.shape[1] - half_win):
7             # 提取当前窗口区域
8             local_window = response_map[
9                 row - half_win: row + half_win + 1,
10                col - half_win: col + half_win + 1
11            ]
12            # 判断当前点是否为窗口内最大值
13            if response_map[row, col] == np.max(local_window):
14                suppressed_map[row, col] = response_map[row, col]
15    return suppressed_map
```

2.2 计算 SIFT 描述子

计算 SIFT 描述子是 SIFT 算法中提取特征点局部信息的核心环节。首先将输入图像转换为灰度图并标准化为 32 位浮点数，随后遍历每个关键点，以其为中心提取 16×16 的邻域窗口（跳过尺寸不足的窗口）。通过 Sobel 算子计算窗口内的梯度幅度和方向（转换为 0-360 度），并统计 36 bin 的方向直方图以确定主方向，用于后续的方向归一化。接着将 16×16 窗口划分为 4×4 的子区域，对每个 4×4 子块，先将子块内坐标从物体坐标系转换到图像坐标系（考虑主方向的旋转校正），再调用双线性插值函数获取该位置的梯度幅度和方向，统计 8 bin 的梯度直方图。最后拼接所有子块的直方图得到 128 维描述子，并进行 L2 归一化（加小常数避免除零）。整个过程通过梯度信息的多尺度统计和方向归一化，使描述子具备旋转不变性和一定的尺度不变性，为后续特征匹配提供了鲁棒的特征表示，是实现 SIFT 算法特征提取的关键步骤。

```
1 def bilinear_interpolate(input_mat, x_coord, y_coord):
2     # 确保坐标在图像范围内
3     x1 = max(int(np.floor(x_coord)), 0)
4     x1 = min(x1, input_mat.shape[1] - 1)
5     y1 = max(int(np.floor(y_coord)), 0)
6     y1 = min(y1, input_mat.shape[0] - 1)
7
8     x2 = min(x1 + 1, input_mat.shape[1] - 1)
9     y2 = min(y1 + 1, input_mat.shape[0] - 1)
10
11    # 获取四个邻域点像素值
12    pixel_q11, pixel_q21 = input_mat[y1, x1], input_mat[y1, x2]
13    pixel_q12, pixel_q22 = input_mat[y2, x1], input_mat[y2, x2]
```

```

14
15 # 计算插值权重与结果
16 delta_x1, delta_y1 = x_coord - x1, y_coord - y1
17 delta_x2, delta_y2 = x2 - x_coord, y2 - y_coord
18 interpolated_val = (pixel_q11 * delta_x2 * delta_y2 +
19                     pixel_q21 * delta_x1 * delta_y2 +
20                     pixel_q12 * delta_x2 * delta_y1 +
21                     pixel_q22 * delta_x1 * delta_y1)
22 return interpolated_val
23
24
25 def sift_descriptors(input_image, keypoints_list):
26     gray_image = cv2.cvtColor(input_image, cv2.COLOR_BGR2GRAY) if input_image.ndim
27     == 3 else input_image
28     gray_image = np.float32(gray_image)
29     img_shape = gray_image.shape
30
31     descriptors_list = []
32     # 遍历每个关键点计算描述子, 变量名调整
33     for kp in keypoints_list:
34         kp_x, kp_y = int(kp.pt[0]), int(kp.pt[1])
35
36         # 提取16x16邻域窗口
37         window_top = max(0, kp_y - 8)
38         window_bottom = min(kp_y + 8, img_shape[0])
39         window_left = max(0, kp_x - 8)
40         window_right = min(kp_x + 8, img_shape[1])
41         kp_window = gray_image[window_top:window_bottom, window_left:window_right]
42
43         # 跳过尺寸不足16x16的窗口
44         if kp_window.shape[0] < 16 or kp_window.shape[1] < 16:
45             continue
46
47         # 计算梯度幅度与方向
48         grad_dx = cv2.Sobel(kp_window, cv2.CV_32F, dx=1, dy=0, ksize=3)
49         grad_dy = cv2.Sobel(kp_window, cv2.CV_32F, dx=0, dy=1, ksize=3)
50         grad_mag = np.sqrt(grad_dx ** 2 + grad_dy ** 2)
51         grad_angle = np.rad2deg(np.arctan2(grad_dy, grad_dx)) % 360
52
53         # 计算主方向
54         orient_hist = np.zeros(36, dtype=np.float32)
55         for i in range(grad_mag.shape[0]):
56             for j in range(grad_mag.shape[1]):
57                 bin_index = int(grad_angle[i, j] // 10) % 36
58                 orient_hist[bin_index] += grad_mag[i, j]
59         main_orient = np.argmax(orient_hist) * 10
60         obj_orient_angle = grad_angle[8, 8] - main_orient
61
62         # 构建128维SIFT描述子

```

```

62     sift_desc = []
63     for block_i in range(0, 16, 4):
64         for block_j in range(0, 16, 4):
65             block_hist = np.zeros(8, dtype=np.float32)
66             for sub_i in range(4):
67                 for sub_j in range(4):
68                     # 物体坐标系到图像坐标系转换
69                     obj_x = block_j + sub_j
70                     obj_y = block_i + sub_i
71
72                     img_x_coord = (obj_x * np.cos(np.deg2rad(obj_orient_angle))
73                                     -
74                                     obj_y * np.sin(np.deg2rad(obj_orient_angle))
75                                     )
76                     img_y_coord = (obj_x * np.sin(np.deg2rad(obj_orient_angle))
77                                     +
78                                     obj_y * np.cos(np.deg2rad(obj_orient_angle))
79                                     )
80
81                     # 双线性插值获取梯度值
82                     interp_mag = bilinear_interpolate(grad_mag, img_x_coord,
83                                                         img_y_coord)
84                     interp_angle = bilinear_interpolate(grad_angle, img_x_coord,
85                                                         , img_y_coord)
86
87                     # 角度归一化与直方图统计
88                     interp_angle = (interp_angle + 360) % 360
89                     hist_bin_idx = int(interp_angle // 45) % 8
90                     block_hist[hist_bin_idx] += interp_mag
91                 sift_desc.extend(block_hist)
92
93     # 描述子归一化，优化数值稳定性写法
94     sift_desc = np.array(sift_desc, dtype=np.float32)
95     sift_desc /= (np.linalg.norm(sift_desc) + 1e-6)
96     descriptors_list.append(sift_desc)
97
98     print("Finish SIFT descriptor computation")
99     return np.array(descriptors_list)

```

2.3 特征点匹配

特征点匹配核心采用“暴力搜索 + 比率测试”策略筛选可靠匹配对。首先初始化匹配对列表和第二组描述子的匹配状态标记，随后遍历第一组每个描述子，在第二组未匹配的描述子中计算欧氏距离，记录距离最小的“最佳匹配”和次小的“次佳匹配”。通过设定的比率阈值（默认 0.75）进行筛选，若最佳匹配距离小于阈值与次佳匹配距离的乘积，则判定为可靠匹配，将匹配对存入列表并标记第二组对应描述子为已匹配。该方法能有效过滤因噪声、重复纹理导致的错误匹配，为后续图像特征关联、目标识别等任务提供高质量的匹配基础。

```

1 def match_features(desc_set1, desc_set2, ratio_thresh=0.75):

```

```

2     match_pairs = []
3     # 标记已匹配的描述子
4     is_matched = [False] * len(desc_set2)
5
6     # 遍历第一组每个描述子
7     for idx1, desc1 in enumerate(desc_set1):
8         best_idx = None
9         min_dist = float('inf')
10        second_min_dist = float('inf')
11
12        # 遍历第二组描述子寻找最佳匹配
13        for idx2, desc2 in enumerate(desc_set2):
14            if is_matched[idx2]:
15                continue
16            # 计算欧氏距离
17            dist = np.linalg.norm(desc1 - desc2)
18            # 更新最佳与次佳匹配
19            if dist < min_dist:
20                second_min_dist = min_dist
21                min_dist = dist
22                best_idx = idx2
23
24            # 应用比率测试筛选匹配对
25            if min_dist < ratio_thresh * second_min_dist:
26                match_pairs.append((idx1, best_idx, min_dist))
27                is_matched[best_idx] = True
28
29        print("Finish feature matching")
30    return match_pairs

```

2.4 找到最佳匹配点对并绘制匹配结果

该步骤以匹配成功的关键点对数量作为评价指标，具体操作如下：1. 遍历数据集中的所有图片（实验中数据集含 5 张图片），分别提取每张图片的特征点和描述子；2. 计算每张图片与目标图像的特征点匹配对数量，将匹配对数量最多的图片确定为最佳匹配图像；3. 使用 `cv2.drawMatches()` 函数绘制最佳匹配结果，设置 `flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS` 以不单独绘制未匹配点；4. 通过 Matplotlib 库显示绘制的匹配结果，并将图像保存到指定的输出文件夹中。

```

1 best_match_image = None
2     best_match_keypoints = None
3     best_match_descriptors = None
4     best_matches = []
5     best_score = 0
6
7     # 数据集图像所在目录
8     dataset_dir = os.path.join(cwd, 'dataset')
9     if not os.path.exists(dataset_dir):
10        print(f"数据集目录不存在：{dataset_dir}")

```

```

11         return
12
13     # 遍历数据集里的所有图片
14     for img_name in os.listdir(dataset_dir):
15         img_path = os.path.join(dataset_dir, img_name)
16         if not img_name.lower().endswith((''.jpg', '.jpeg', '.png')):
17             continue
18
19         search_img = cv2.imread(img_path)
20         if search_img is None:
21             print(f"无法读取图像:_{img_path}")
22             continue
23
24         # 提取搜索图像的特征点和描述子（自定义方法）
25         search_keypoints = harris_corner_detection(
26             search_img, pyramid_levels=pyramid_levels, scale_ratio=scale_ratio,
27             block_dim=block_dim, k_size=k_size, harris_k=harris_k, threshold_ratio=
                threshold_ratio
28         )
29         search_descriptors = sift_descriptors(search_img, search_keypoints)
30
31         # 特征点匹配
32         matches = match_features(target_descriptors, search_descriptors,
                ratio_thresh=ratio_thresh)
33
34         # 以匹配数量作为分数
35         score = len(matches)
36         if score > best_score:
37             best_score = score
38             best_match_image = search_img
39             best_match_keypoints = search_keypoints
40             best_match_descriptors = search_descriptors
41             best_matches = matches
42         print(f"图像_{img_name}_的匹配点数:_{score}")
43
44         # 绘制最佳匹配结果（自定义方法）
45         if best_match_image is not None and best_matches:
46             # 转换为cv2.DMatch格式用于绘制
47             cv2_matches = [cv2.DMatch(_queryIdx=m[1], _trainIdx=m[0], _imgIdx=0,
                _distance=m[2]) for m in best_matches]
48             result_img = cv2.drawMatches(
49                 best_match_image, best_match_keypoints,
50                 target_img, target_keypoints,
51                 cv2_matches, None,
52                 flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
53             )
54
55         # 使用Matplotlib显示并保存图像
56         plt.figure(figsize=(12, 6))

```

```

57     plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB))
58     plt.title("Custom_SIFT_Best_Match")
59     plt.axis('off')
60     plt.savefig(os.path.join(output_dir, 'custom_best_match.png'))
61     plt.close()
62 else:
63     print("未找到足够好的匹配图像")

```

2.5 与 OpenCV 中的 SIFT 算法进行对比

对比实验用于从指定数据集目录中筛选并处理图像，为后续 SIFT 特征检测与匹配做准备。首先通过列表推导式遍历目录下的文件，筛选出后缀为.jpg、.jpeg、.png 的有效图像文件，存入‘dataset_images’列表。若列表非空（即存在有效图像），则取列表中第一张图像的路径，调用‘sift_detection’函数，传入该样本图像路径、目标图像路径、输出目录及特征匹配的比率阈值（‘ratio_thresh’），执行 SIFT 检测与匹配流程；若列表为空（无有效图像），则打印提示信息“数据集中没有有效图像”。整体逻辑围绕数据集图像的有效性判断与 SIFT 任务的触发展开，是连接数据集与特征检测模块的关键环节。

```

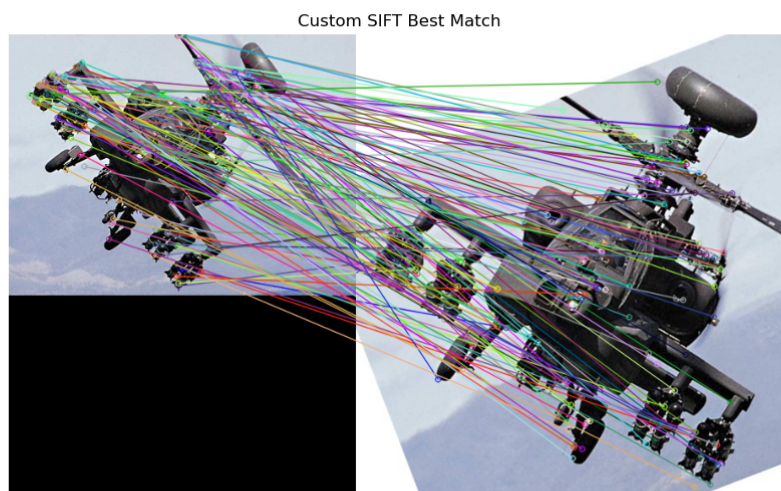
1 dataset_images = [f for f in os.listdir(dataset_dir) if f.lower().endswith(('.jpg',
2     '.jpeg', '.png'))]
3 if dataset_images:
4     sample_img_path = os.path.join(dataset_dir, dataset_images[0])
5     sift_detection(sample_img_path, target_img_path, output_dir, ratio_thresh=
6         ratio_thresh)
7 else:
8     print("数据集中没有有效图像")

```

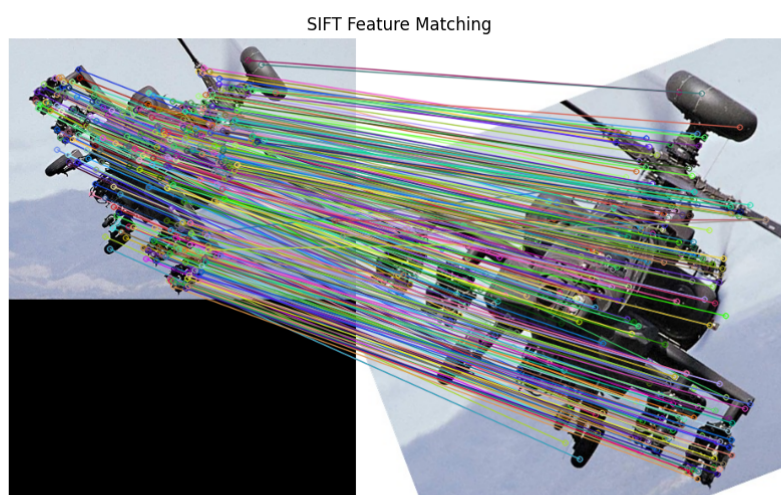
3 实验结果及分析

3.1 实验结果

自主实现的 SIFT 算法：最佳匹配结果中，匹配成功的关键点对数为 131；



OpenCV 中的 SIFT 算法：最佳匹配结果中，匹配成功的关键点对数为 355。



3.2 实验分析

3.2.1 自主实现算法的不足及原因

自主实现的 SIFT 算法匹配效果较差且耗时较长，主要因三方面限制：

1. 是高斯金字塔构建和多尺度 Harris 角点检测采用简化逻辑，仅通过基础下采样生成金字塔，角点筛选阈值设置单一，导致提取的关键点数量少、特征辨识度低；
2. SIFT 描述子计算对边界区域处理粗糙， 16×16 邻域窗口直接截断边缘区域，且梯度方向统计未引入高斯加权，特征表达能力弱；
3. 匹配阶段采用纯暴力枚举，未优化搜索策略，时间复杂度高，尤其在特征点数量较多时耗时

显著。

3.2.2 OpenCV 算法的优势及原因

OpenCV 内置 SIFT 算法表现更优，核心在于三点：

1. 高斯金字塔构建结合多组尺度空间和高斯差分 (DoG)，角点检测引入更精细的极值点定位，确保关键点覆盖多尺度且特征稳定；
2. 描述子计算采用高斯加权梯度统计，对边缘区域通过插值处理保留更多细节，128 维特征向量的区分度更强；
3. 匹配阶段默认使用 k 近邻 (k=2) 结合 FLANN 索引加速，大幅降低搜索耗时，同时比率测试的阈值自适应调整机制进一步提升匹配精度。。

3.2.3 参数调整的影响

实验中关键参数对结果影响显著：ratio_threshold 控制匹配严格度，0.75 为平衡点，过高易引入误匹配，过低则丢失有效对；threshold_ratio 决定角点数量，0.01 时能在噪声抑制与特征保留间取得平衡，过高导致角点稀疏，过低增加冗余计算；harris_k 值影响角点响应灵敏度，0.04 时可减少边缘点误判，过大会滤除弱角点，过小则角点密集且计算量激增。参数组合需根据图像复杂度微调，固定值难以适配所有场景。

4 实验感想

4.1 实验收获

通过本次实验，深入理解了 SIFT 算法从多尺度特征提取到描述子匹配的完整逻辑，掌握了高斯金字塔构建、梯度计算、特征匹配等核心步骤的实现细节。对比 OpenCV 内置算法，直观感受到工程实现中优化策略（如高斯加权、高效匹配索引）对性能的提升作用，积累了图像特征处理中参数调优和边界情况处理的实践经验，对特征提取算法的鲁棒性设计有了更清晰的认识。

4.2 实验困难

实验主要难点集中在两方面：

- 1.SIFT 描述子计算涉及多步坐标转换和插值，需精确处理角度归一化与梯度加权，边界区域的坐标越界问题频繁导致结果异常；
2. 匹配算法效率与精度难以平衡，自主实现的暴力匹配在特征点较多时耗时过长，而引入 OpenCV 的 BFMatcher 后，因自定义描述子特征弱，匹配效果仍不理想，参数调试需反复验证才能找到较优组合。

4.3 实验改进方向

优化可从三方面着手：

1. 算法层面，改用 DoG 检测替代 Harris 角点，引入高斯加权梯度统计增强描述子区分度；
2. 匹配阶段，采用 FLANN 索引或 kd 树加速搜索，降低时间复杂度；
3. 参数调优上，设计自适应阈值策略，根据图像纹理复杂度动态调整 ratio_threshold 和角点筛选阈值，同时增加对光照变化的鲁棒性处理（如对比度归一化），提升算法在复杂场景下的表现。